Inventory Management System for B2B SaaS

Part 1: Code Review & Debugging:

Errors found:

1. No error handling: if any required field is missing, it throws an unhandled KeyError and returns a 500.
2. Two separate db.session.commit() calls: If the second commit fails, the product is already saved. So, now we have a product with no inventory record.
3. No authentication or authorization: Any unauthorized caller can create products.
4. Price stored without type safety: data['price'] is passed directly. If someone sends a string like 'free', it'll either error silently or store bad data depending on the ORM config.

CORRECT CODE:

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # 1. Validate required fields
    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    for field in required_fields:
        if field not in data:
            return {"error": f"Missing required field: {field}"}, 400

    # 2. Validate types
    try:
        price = float(data['price'])
        initial_quantity = int(data['initial_quantity'])
        warehouse_id = int(data['warehouse_id'])
    except (ValueError, TypeError):
        return {"error": "Invalid data types for price, quantity, or warehouse_id"}, 400

    if price < 0 or initial_quantity < 0:
        return {"error": "Price and quantity must be non-negative"}, 400

    try:
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=price,
            warehouse_id=warehouse_id
        )
        db.session.add(product)
        db.session.flush()  # gets product.id without committing yet
```

```
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=warehouse_id,
            quantity=initial_quantity
        )
        db.session.add(inventory)

        db.session.commit()

    except Exception as e:
        db.session.rollback()
        return {"error": "Failed to create product", "details": str(e)}, 500

    return {"message": "Product created", "product_id": product.id}, 201
```

Part 2: Database Design:

```
-- Core company/tenant
CREATE TABLE companies (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    created_at  TIMESTAMP DEFAULT NOW()
);

-- Warehouses belong to a company
CREATE TABLE warehouses (
    id          SERIAL PRIMARY KEY,
    company_id  INT NOT NULL REFERENCES companies(id),
    name        VARCHAR(255) NOT NULL,
    location    VARCHAR(255),
    created_at  TIMESTAMP DEFAULT NOW()
);

-- Products are platform-wide; SKU is unique
CREATE TABLE products (
    id          SERIAL PRIMARY KEY,
    company_id   INT NOT NULL REFERENCES companies(id),
    name         VARCHAR(255) NOT NULL,
    sku          VARCHAR(100) NOT NULL UNIQUE,
    price        NUMERIC(10, 2) NOT NULL,
    product_type VARCHAR(50),        -- e.g. 'standard', 'bundle'
    low_stock_threshold INT DEFAULT 10,
    created_at   TIMESTAMP DEFAULT NOW()
);

CREATE TABLE inventory (
```

```
    id          SERIAL PRIMARY KEY,
    product_id   INT NOT NULL REFERENCES products(id),
    warehouse_id INT NOT NULL REFERENCES warehouses(id),
    quantity     INT NOT NULL DEFAULT 0,
    updated_at   TIMESTAMP DEFAULT NOW(),
    UNIQUE(product_id, warehouse_id)
);

-- Audit log
CREATE TABLE inventory_logs (
    id          SERIAL PRIMARY KEY,
    product_id   INT NOT NULL REFERENCES products(id),
    warehouse_id INT NOT NULL REFERENCES warehouses(id),
    change_qty   INT NOT NULL,        -- positive = restock, negative = sale
    reason       VARCHAR(50),         -- 'sale', 'restock', 'adjustment'
    created_at   TIMESTAMP DEFAULT NOW()
);

-- Suppliers
CREATE TABLE suppliers (
    id           SERIAL PRIMARY KEY,
    name         VARCHAR(255) NOT NULL,
    contact_email VARCHAR(255),
    phone        VARCHAR(50),
    created_at    TIMESTAMP DEFAULT NOW()
);

-- Which supplier provides which product
CREATE TABLE product_suppliers (
    id          SERIAL PRIMARY KEY,
    product_id  INT NOT NULL REFERENCES products(id),
    supplier_id INT NOT NULL REFERENCES suppliers(id),
    unit_cost   NUMERIC(10, 2),
    lead_days   INT,                  -- average restock lead time
    is_primary  BOOLEAN DEFAULT FALSE,
    UNIQUE(product_id, supplier_id)
);

CREATE TABLE bundle_items (
    id             SERIAL PRIMARY KEY,
    bundle_id       INT NOT NULL REFERENCES products(id),
    component_id    INT NOT NULL REFERENCES products(id),
    quantity        INT NOT NULL DEFAULT 1,
    UNIQUE(bundle_id, component_id)
);

CREATE INDEX idx_inventory_product    ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse  ON inventory(warehouse_id);
```

```
CREATE INDEX idx_inventory_logs_time  ON inventory_logs(created_at);
CREATE INDEX idx_products_company     ON products(company_id);
```

Part 3: API Implementation

Assumptions Made

- "Recent sales activity" = at least one sale in the last 30 days (from `inventory_logs`)
- Low stock threshold is stored on the `products` table
- `days_until_stockout` is calculated as `current_stock / avg_daily_sales` (based on last 30 days)
- Primary supplier is returned; if none marked primary, return the first one
- A company can have many warehouses; we alert per product-per-warehouse

CODE:

```
from flask import jsonify
from datetime import datetime, timedelta
from sqlalchemy import func
from sqlalchemy.orm import joinedload

@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
def low_stock_alerts(company_id):

    if not Company.query.get(company_id):
        return jsonify({"error": "Company not found"}), 404

    thirty_days_ago = datetime.utcnow() - timedelta(days=30)

    # Aggregate sales per product/warehouse in last 30 days
    recent_sales = (
        db.session.query(
            InventoryLog.product_id,
            InventoryLog.warehouse_id,
            func.sum(func.abs(InventoryLog.change_qty)).label('total_sold')
        )
        .join(Warehouse, InventoryLog.warehouse_id == Warehouse.id)
        .filter(
            Warehouse.company_id == company_id,        # scoped to company directly
            InventoryLog.reason == 'sale',
            InventoryLog.created_at >= thirty_days_ago
        )
        .group_by(InventoryLog.product_id, InventoryLog.warehouse_id)
        .subquery()
```

```
    )

    # Single query: inventory + product + warehouse + sales, filtered to low stock
    results = (
        db.session.query(Inventory, Product, Warehouse, recent_sales.c.total_sold)
        .join(Product, Inventory.product_id == Product.id)
        .join(Warehouse, Inventory.warehouse_id == Warehouse.id)
        .join(recent_sales, (recent_sales.c.product_id == Inventory.product_id) &
                            (recent_sales.c.warehouse_id == Inventory.warehouse_id))
        .filter(
            Warehouse.company_id == company_id,
            Inventory.quantity < Product.low_stock_threshold
        )
        .options(joinedload(Product.suppliers))        # avoids N+1 supplier queries
        .all()
    )

    def build_alert(inventory, product, warehouse, total_sold):
        avg_daily = total_sold / 30.0
        supplier = next((s for s in product.suppliers if s.is_primary),
                        next(iter(product.suppliers), None))
        return {
            "product_id": product.id,
            "product_name": product.name,
            "sku": product.sku,
            "warehouse_id": warehouse.id,
            "warehouse_name": warehouse.name,
            "current_stock": inventory.quantity,
            "threshold": product.low_stock_threshold,
            "days_until_stockout": int(inventory.quantity / avg_daily) if avg_daily else None,
            "supplier": {"id": supplier.id, "name": supplier.name, "contact_email":
supplier.contact_email} if supplier else None
        }

    alerts = [build_alert(*row) for row in results]
    return jsonify({"alerts": alerts, "total_alerts": len(alerts)}), 200
```